

OUTPUTS

Problem 1: Array Creation

1. Empty array (2x2):

```
[[23. 51.]  
 [13. 29.]]
```

2. All ones array (4x2):

```
[[1. 1.]  
 [1. 1.]  
 [1. 1.]  
 [1. 1.]]
```

3. Full array (3x3) with 7:

```
[[7 7 7]  
 [7 7 7]  
 [7 7 7]]
```

4. Zeros like ref_array:

```
[[0 0]  
 [0 0]]
```

5. Ones like ref_array:

```
[[1 1]  
 [1 1]]
```

6. Converted list to array:

```
[1 2 3 4]
```

Problem 2: Array Manipulation

1. Array from 10 to 49:

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33  
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

2. 3x3 matrix from 0 to 8:

```
[[0 1 2]  
 [3 4 5]  
 [6 7 8]]
```

3. Identity matrix (3x3):

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

4. Random array (size 30):

```
[0.16706618 0.51391568 0.63129795 0.60159322 0.35982205 0.8907916  
0.92060649 0.6981321 0.95710001 0.36257303 0.60979088 0.95407398  
0.15017943 0.60089691 0.39780383 0.95193142 0.18889946 0.80761812  
0.49830679 0.77588445 0.22413673 0.35699578 0.70306652 0.18423196  
0.56246619 0.78776215 0.42719555 0.94282907 0.78589611 0.4647441 ]
```

Mean of array: 0.582586924771185

5. Random 10x10 array:

```
[[0.8444858 0.07496715 0.92749832 0.54203838 0.38869024 0.28473211  
0.70649358 0.71349418 0.10589333 0.2290473 ]  
[0.45245374 0.15236727 0.1364742 0.89721691 0.77954977 0.20476386  
0.61255058 0.50249168 0.56400069 0.63058875]  
[0.91535714 0.79207607 0.39067805 0.49732774 0.0566175 0.86800899  
0.29446932 0.07021034 0.43602239 0.37052373]  
[0.08238335 0.46716609 0.41501988 0.43982908 0.0930381 0.81073065  
0.22599836 0.65739883 0.01318748 0.31330848]  
[0.46906126 0.87201864 0.01555936 0.73413381 0.87091802 0.99895142  
0.73519427 0.01584822 0.10719978 0.8694453 ]  
[0.3653174 0.5902674 0.64198414 0.92138045 0.22290999 0.25246328  
0.0613143 0.8471525 0.42101728 0.41181062]  
[0.23299425 0.26966542 0.6297988 0.46024863 0.88929424 0.15592121  
0.25342516 0.11237906 0.43328881 0.24608083]  
[0.05589514 0.43261352 0.33691518 0.22564799 0.38431757 0.45522187  
0.07162075 0.48927254 0.65397312 0.0282972 ]  
[0.94519718 0.52332328 0.53853452 0.77249375 0.1394855 0.8815471  
0.04872344 0.80882752 0.93572068 0.78432263]  
[0.96147333 0.46693141 0.87608518 0.3871131 0.35166243 0.209996  
0.87006485 0.44204871 0.97661177 0.00940338]]
```

Min: 0.009403375541012005 Max: 0.9989514170266361

```

6. Zero array with 5th element = 1:
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]

7. Reversed array:
[0, 4, 0, 0, 2, 1]

8. 2D array with 1 on border and 0 inside:
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]

9. 8x8 checkerboard pattern:
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]

```

Problem 3: Array Operations

x:

```
[[1 2]
 [3 5]]
```

y:

```
[[5 6]
 [7 8]]
```

v: [9 10]

w: [11 12]

1. x + y:

```
[[ 6  8]
 [10 13]]
```

2. x - y:

```
[[ -4 -4]
 [-4 -3]]
```

3. x * 3:

```
[[ 3  6]
 [ 9 15]]
```

4. Square of x:

```
[[ 1  4]
 [ 9 25]]
```

5. Dot products:

v . w = 219

x . v =

```
[29 77]
```

x . y =

```
[[19 22]
 [50 58]]
```

6. Concatenate x and y along rows:

```
[[1 2]
 [3 5]
 [5 6]
 [7 8]]
```

Concatenate v and w along columns:

```
[[ 9 10]
 [11 12]]
```

7. Concatenate x and v:

Error: all the input arrays must have same number of dimensions, but the array at index 0 has 2 dimension(s) and the array at index 1 has 1 dimension(s)

Problem 4: Matrix Operations

1. $A * A_{inv}$:

```
[[1.00000000e+00 4.4408921e-16]
 [0.00000000e+00 1.0000000e+00]]
```

2. AB :

```
[[23 13]
 [51 29]]
```

BA :

```
[[36 44]
 [13 16]]
```

$AB == BA$? False

3. $(AB)^T$:

```
[[23 51]
 [13 29]]
```

$B^T * A^T$:

```
[[23 51]
 [13 29]]
```

$(AB)^T == B^T * A^T$? True

Solving Linear Equations

Solution $[x, y, z]$: [2. 1. -2.]

1. Element-wise Addition

Python List Addition Time: 0.08253169059753418 seconds

NumPy Array Addition Time: 0.0009999275207519531 seconds

2. Element-wise Multiplication

Python List Multiplication Time: 0.10497736930847168 seconds

NumPy Array Multiplication Time: 0.0020051002502441406 seconds

3. Dot Product

Python List Dot Product Time: 0.2013871669769287 seconds

NumPy Array Dot Product Time: 0.0009996891021728516 seconds